pfi

Python for Fixed Income

COURSE 2 · BOND MATH

Duration and DV01

WEEK 2 · DAY 4

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

The desk lead's note: a basis point in dollars

The Desk Lead's Note

We hold 5,000,000 Quorvane, 3,000,000 Dunmarrow, 4,000,000 Pellwick and 2,000,000 Halvering, settling Monday 30 November. If yields move one basis point, what does each position make or lose in dollars, and the book? Give "bondmath" duration and DV01, tied to Excel. Last, September's holdings file has a "duration" column that does not say which duration it is: find out.

A basis point (bp) is a hundredth of a percent, as in notebook 03. The issuers and positions are invented; every statement describes the data.

The Price Every Sensitivity Starts From

issuer	par_held	dirty
Quorvane Energy	5000000	100.236551
Dunmarrow Rail	3000000	101.010586
Pellwick Paper	4000000	101.179914
Halvering Foods	2000000	99.872491

Quorvane's 100.236551 is notebook 08's 99.361551 quote plus 0.875 of accrued.

```
.....................   [100%]
23 passed in 0.50s
```

Every sensitivity today is measured on the dirty price: that is what the position is worth.

Notebook 08 priced these bonds on 30 November; today asks how far each price moves.

9.1 Macaulay duration: the average time

Quorvane matures on 30 September 2031, but that is only the date of its last payment. Nine coupons come before it.

```
days_accrued, days_in_period, days_to_next = bondmath.coupon_period_days(settlement,
                                                                         quorvane_maturity)
w = days_to_next / days_in_period
rows = []
for k in range(10):
    cash_flow = 5.25 / 2 + (100 if k == 9 else 0)
    rows.append({'time_years': (w + k) / 2, 'pv': cash_flow / (1 + 5.40 / 100 / 2) ** (w + k)})
flows = pd.DataFrame(rows)
flows['weight'] = flows['pv'] / flows['pv'].sum()
flows.style.format('{:.6f}').background_gradient(subset=['weight'], cmap='Greens')
```

The weights add up to 1; the last payment, coupon plus face, carries 0.791 of the value.

That average is Macaulay duration: each payment's time, weighted by its present value over their sum, the dirty price.

Macaulay, 1938

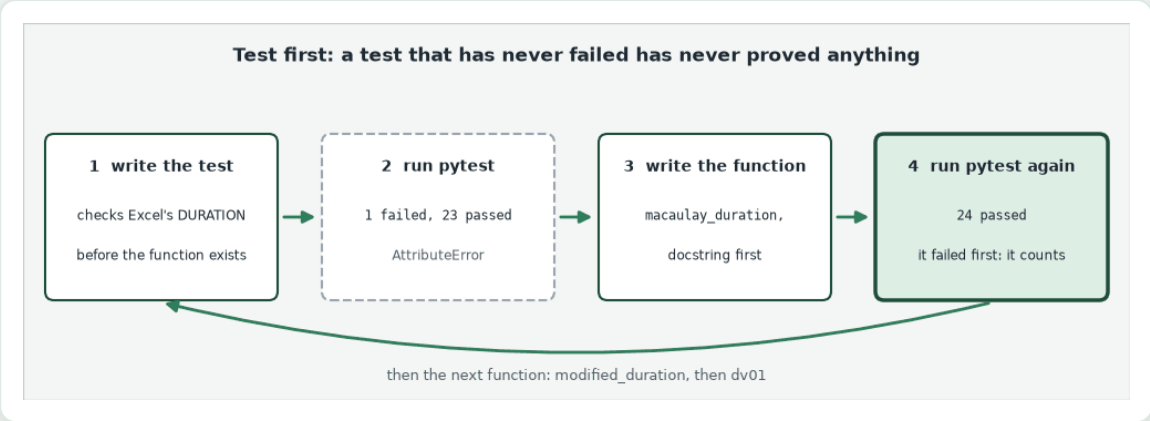
In 1938 Frederick R. Macaulay, writing for the National Bureau of Economic Research, argued that maturity, the date of the last payment, is a poor measure of how long a loan really lasts: "'Duration' is a reality of which 'maturity' is only one factor." He weighted each payment's time by its present value over their sum, "the price paid for the bond": the cell above.

Excel side by side. A blank sheet with k = 0 to 9 typed in A2:A11.

Job	Python	Excel
Times, cash flows, present values, B2 to D2 filled down	`flows`	=(120/180+A2)/2, =IF(A2=9,5.25/2+100,5.25/2), =C2/(1+5.4/100/2)^(120/180+A2)
The weighted average time	`macaulay_by_hand`	=SUMPRODUCT(B2:B11,D2:D11)/SUM(D2:D11)

Story source: NBER, https://www.nber.org/books-and-chapters/some-theoretical-problems-suggested-movements-interest-rates-bond-yields-and-stock-prices-united/concept-long-term-interest-rates

9.2 Test first: macaulay_duration



Every function so far was written first and tested after. Today the test comes first: run before its function exists, it must fail, and a test that has never failed has never proved anything.

"macaulay_duration" will take price's six arguments. The test holds it to Excel's "duration" column on all 306 reference rows; "bond_inputs" (notebook 07) turns a row into the six arguments.

```
%%writefile -a test_bondmath.py
def test_macaulay_matches_excel_duration():
    for row in REFERENCE:
        result = bondmath.macaulay_duration(*bond_inputs(row))
        assert result == pytest.approx(float(row["duration"]), abs=1e-6), row["case_id"]
```

Run It Before the Function Exists

```
......................F   [100%]
E   AttributeError: no attribute 'macaulay_duration'
1 failed, 23 passed in 0.50s
```

The "E" line says why: "bondmath" has no "macaulay_duration" yet. That is the failure we wanted.

Write the function, and run the test again

```
%%writefile -a bondmath.py
def macaulay_duration(settlement: date, maturity: date, coupon_pct: float, yield_pct: float, frequency: int = 2,
                      basis: str = "30/360") -> float:
    dates = coupon_dates(settlement, maturity, frequency)
    _, days_in_period, days_to_next = coupon_period_days(settlement, maturity, frequency, basis)
    coupons_left = len(dates) - 1
    w = days_to_next / days_in_period
    period_yield = yield_pct / 100 / frequency
    coupon = coupon_pct / frequency
    total_pv, weighted_time = 0.0, 0.0
    for k in range(coupons_left):
        cash_flow = coupon + 100 if k == coupons_left - 1 else coupon
        time_years = (w + k) / frequency
        pv = cash_flow / (1 + period_yield) ** (w + k)
        total_pv += pv
        weighted_time += time_years * pv
    return weighted_time / total_pv
```

The docstring states the units (years), the convention (t = (w + k) / f, weighted by present value) and "DURATION" with its "/100". The loop is 9.1's, with the coupon, yield and frequency as arguments.

```
........................   [100%]
24 passed in 0.50s
```

The test that failed now passes, on all 306 rows.

Three Bonds Against DURATION

issuer	years_to_maturity	macaulay	excel
Dunmarrow Rail	6.833676	5.629619	5.629619
Pellwick Paper	2.833676	2.665045	2.665045
Halvering Foods	9.834360	7.020017	7.020017

The gap runs from 0.17 years for Pellwick to 2.81 for Halvering, whose 7.5 percent coupon brings value forward.

Job	Excel
Quorvane	=DURATION(DATE(2026,11,30),DATE(2031,9,30),5.25/100,5.4/100,2,0)
The zero	=DURATION(DATE(2026,11,30),DATE(2031,9,30),0,5.4/100,2,0)

9.3 Modified duration: price sensitivity

Macaulay duration is a time; the desk asks how far the price moves. Divide it by one plus the period yield and you have modified duration, the price's slope as a share of the price:

modified duration = Macaulay duration / (1 + y / 2), with y the yield as a decimal and 2 coupons a year.

The price moves about modified duration percent for 100 bp, and modified duration x 0.0001, as a share, for 1 bp.

```
modified_by_hand = macaulay_by_hand / (1 + 5.40 / 100 / 2)
```

One Basis Point, Repriced

```
dirty_now = bondmath.dirty_price(settlement, quorvane_maturity, 5.25, 5.40)
dirty_up = bondmath.dirty_price(settlement, quorvane_maturity, 5.25, 5.41)
print(f'Repriced:          {(dirty_up / dirty_now - 1) * 100:.8f} percent')
print(f'Modified duration: {-modified_by_hand * 0.0001 * 100:.8f} percent')
```

The straight line and the reprice agree to 1e-05 percent of the price. How far the line misses for 100 bp is notebook 10's question.

Test and function, against MDURATION

The test reads Excel's "mduration" column. The function divides "macaulay_duration" by one plus the period yield rather than repeat the loop.

```
%%writefile -a test_bondmath.py
def test_modified_matches_excel_mduration():
    for row in REFERENCE:
        result = bondmath.modified_duration(*bond_inputs(row))
        assert result == pytest.approx(float(row["mduration"]), abs=1e-6), row["case_id"]
%%writefile -a bondmath.py
def modified_duration(settlement: date, maturity: date, coupon_pct: float, yield_pct: float, frequency: int = 2,
                      basis: str = "30/360") -> float:
    period_yield = yield_pct / 100 / frequency
    return macaulay_duration(settlement, maturity, coupon_pct, yield_pct, frequency, basis) / (1 + period_yield)
```

```
.........................   [100%]
25 passed in 0.50s
```

Three Bonds Against MDURATION

issuer	yield_pct	modified	excel
Dunmarrow Rail	6.125	5.462335	5.462335
Pellwick Paper	4.600	2.605127	2.605127
Halvering Foods	7.700	6.759766	6.759766

Each is its Macaulay duration over 1 + y / 2: 1.023 for Pellwick up to 1.0385 for Halvering.

9.4 DV01 per 100 and per position

A trader talks in price points and a portfolio manager in dollars. DV01, the dollar value of a basis point, is modified duration x dirty price x 0.0001: price points per 100 of face that a long position gains when the yield falls 1 bp. A position's DV01 in dollars is DV01 x par held / 100.

```
%%writefile -a test_bondmath.py
def test_dv01_matches_excel_formula():
    for row in REFERENCE:
        result = bondmath.dv01(*bond_inputs(row))
        assert result == pytest.approx(float(row["dv01"]), abs=1e-8), row["case_id"]
%%writefile -a bondmath.py
def dv01(settlement: date, maturity: date, coupon_pct: float, yield_pct: float, frequency: int = 2,
         basis: str = "30/360") -> float:
    duration_years = modified_duration(settlement, maturity, coupon_pct, yield_pct, frequency, basis)
    full_price = dirty_price(settlement, maturity, coupon_pct, yield_pct, frequency, basis)
    return duration_years * full_price * 0.0001
```

Excel has no DV01 function: the reference file's "dv01" column is "MDURATION" x ("PRICE" + accrued) / 10000, so the test is tighter, 1e-8 per 100.

issuer	par_held	dv01	excel
Dunmarrow Rail	3000000	0.055175	0.055175
Pellwick Paper	4000000	0.026359	0.026359
Halvering Foods	2000000	0.067511	0.067511

Job	Excel
Clean price, B1	=PRICE(DATE(2026,11,30),DATE(2031,9,30),5.25/100,5.4/100,100,2,0)
Accrued, B2	=100*5.25/100/2*COUPDAYBS(DATE(2026,11,30),DATE(2031,9,30),2,0)/COUPDAYS(DATE(2026,11,30),DATE(2031,9,30),2,0)
Modified duration, B4	=MDURATION(DATE(2026,11,30),DATE(2031,9,30),5.25/100,5.4/100,2,0)
DV01 per 100, B5	=B4*(B1+B2)/10000

Halvering, the longest bond, moves most per 100 of face.

9.5 Checking DV01 by bumping the yield

"dv01" is a slope at 5.40. A check that needs no formula: price the bond 1 bp lower and 1 bp higher and take half the difference, the slope of the chord between them.

Price It 1 bp Either Side

```
bumps = bonds[['issuer', 'dv01']].copy()
bumps['bump'] = [(bondmath.price(settlement, m, c, y - 0.01)
                  - bondmath.price(settlement, m, c, y + 0.01)) / 2
                 for m, c, y in zip(maturities, coupons, yields)]
bumps['gap'] = bumps['bump'] - bumps['dv01']
bumps
```

issuer	dv01	bump	gap
Dunmarrow Rail	0.055175	0.055175	4.503757e-09
Pellwick Paper	0.026359	0.026359	5.215240e-10
Halvering Foods	0.067511	0.067511	9.715958e-09

The bump uses clean prices and "dv01" the dirty price; accrued depends on the dates, not the yield, so it cancels in the difference.

```
%%writefile -a test_bondmath.py
def test_dv01_close_to_bump():
    rows = [row for row in REFERENCE if row["coupnum"] != "1"]
    for row in rows:
        bump = (float(row["price_down_1bp"]) - float(row["price_up_1bp"])) / 2
        assert bondmath.dv01(*bond_inputs(row)) == pytest.approx(bump, abs=1e-6), row["case_id"]
```

Excel side by side. The bump with two "PRICE" cells.

Job	Python	Excel
1 bp lower, B8	`bondmath.price(settlement, quorvane_maturity, 5.25, 5.39)`	=PRICE(DATE(2026,11,30),DATE(2031,9,30),5.25/100,(5.4-0.01)/100,100,2,0)
1 bp higher, B9	`bondmath.price(settlement, quorvane_maturity, 5.25, 5.41)`	=PRICE(DATE(2026,11,30),DATE(2031,9,30),5.25/100,(5.4+0.01)/100,100,2,0)
Half the difference, B10	`bumps['bump']`	=(B8-B9)/2

9.6 Commit, then look back

The module is done for the day. Commit it, then ask Git a new question: when did a file change?

```
!git -C "{work}" status --short
!git -C "{work}" diff --stat
!git -C "{work}" add bondmath.py test_bondmath.py
message = 'Add macaulay_duration, modified_duration, and dv01, tested against Excel'
!git -C "{work}" commit -q -m "{message}"
 bondmath.py   | 55 +++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++
 test_bondmath.py | 28 ++++++++++++++++++++++++++++
 2 files changed, 83 insertions(+)
```

```
!git -C "{work}" log --oneline
74945cb Add macaulay_duration, modified_duration, and dv01, tested against Excel
20b3546 Notebook 08: price, dirty_price, and yield_to_maturity
98124d3 Notebook 07: days_30_360, coupon_period_days, and accrued_interest
9fe8f04 Notebook 06: add_months, coupon_dates, and a .gitignore
322960d Notebook 05: tests of par pricing and of price against yield
7147a11 Notebook 04: yield_from_price
3f57cac Notebook 03: convert_yield
408a678 Notebook 02: cash_flows and price_from_yield
d029474 Notebook 01: future_value and present_value
```

"HEAD" is the newest commit, "HEAD~1" the one before it, "HEAD~2" two back.

```
!git -C "{work}" diff HEAD~1 -- bondmath.py
@@ -363,3 +363,58 @@ def yield_to_maturity(settlement: date, maturity: date, coupon_pct: float, clean
    high_pct = mid_pct
    # the middle of the final bracket
    return (low_pct + high_pct) / 2
+
+
```

Every line starts with "+": today added three functions and changed nothing that was there. Git answers "when did this change?" for one file, and "HEAD~1" names the commit before.

9.7 Signs: long and short

Price and yield move opposite ways, so say the convention out loud:

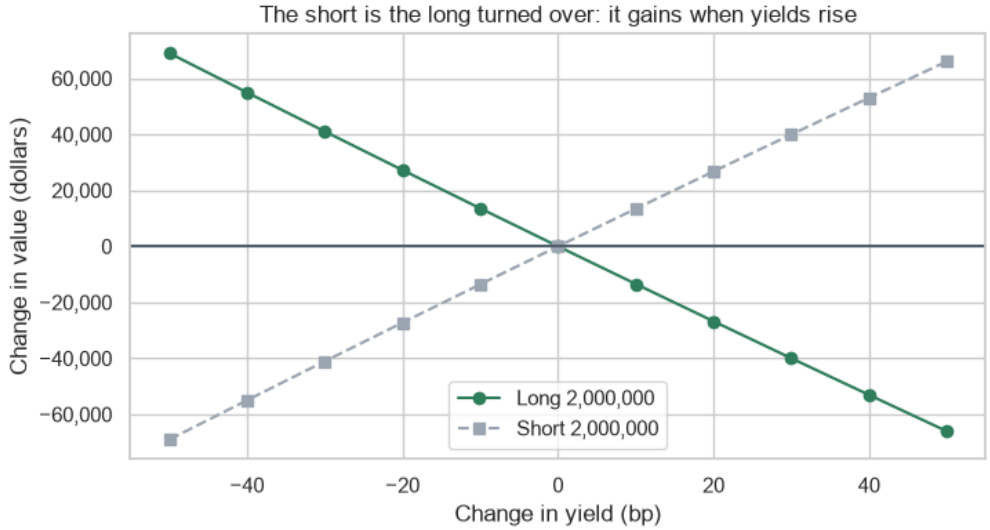
DV01 is the gain on a long position when the yield falls 1 bp; it loses about as much when the yield rises 1 bp.

For a move of n bp, the change in value is about minus position DV01 times n.

A short has a negative par held, so a negative DV01: it gains when yields rise.

```
Short DV01: -1,350.23 dollars per basis point
Change for a 1 bp rise: 1,350.23 dollars
```

Same formula, opposite sign. Some systems flip the sign and report the change for a rise: check the convention before adding DV01s from two sources.



Excel side by side: the short, one cell

```
=MDURATION(DATE(2026,11,30),DATE(2036,9,30),7.5/100,7.7/100,2,
0)*(PRICE(DATE(2026,11,30),DATE(2036,9,30),7.5/100,7.7/100,100,2,
0)+100*7.5/100/2*COUPDAYBS(DATE(2026,11,30),DATE(2036,9,30),2,
0)/COUPDAYS(DATE(2026,11,30),DATE(2036,9,30),2,0))/10000*-2000000/100
```

The short, one cell: -1350.2293587337144

9.8 Which duration is the file's column?

September's holdings file has a "duration" column, rounded to 3 decimals, for 200 bonds on 30 September. Its header does not say which duration; Course 1's Series and DataFrames notebook listed it as modified.

Both Durations, All 200 Bonds

```
holdings = pd.read_csv('holdings.csv')
as_of = pd.to_datetime(holdings['as_of_date']).dt.date
maturity_dates = pd.to_datetime(holdings['maturity']).dt.date
inputs = list(zip(as_of, maturity_dates, holdings['coupon'], holdings['yield_pct']))
holdings['macaulay_python'] = [bondmath.macaulay_duration(*bond) for bond in inputs]
holdings['modified_python'] = [bondmath.modified_duration(*bond) for bond in inputs]
```

Within Rounding on Every Bond

```
holdings['gap'] = (holdings['modified_python'] - holdings['duration']).abs()
assert holdings['gap'].max() <= 0.0005 + 1e-9
same = (holdings['modified_python'].round(3) == holdings['duration']).sum()
print(same, 'of', len(holdings), 'bonds round to the file value')
200 of 200 bonds round to the file value
```

Unrounded, "test_modified_matches_excel_mduration" already ties all 200 to Excel's "MDURATION": they are 200 of the reference rows.

The file's "duration" column is modified duration, on all 200 bonds.

Bring it home, and what you can now do

Copy the work folder's two files into your own "my_bondmath" folder, test them there, and commit. The copy replaces your files, so anything of your own must be in the work folder first.

```
Set-Location "$HOME\projects\my_bondmath"
Copy-Item "$env:TEMP\pfi_course2_09_k4w7q2xr\bondmath.py", "$env:TEMP\pfi_course2_09_k4w7q2xr\test_bondmath.py" .
python -m pytest -q
git diff --stat
git commit -am "Add macaulay_duration, modified_duration, and dv01, tested against Excel"
git log --oneline -- bondmath.py
git diff HEAD~1 -- bondmath.py
```

pytest shows 27 passed, and the log lists every day that changed your module. A long log or diff opens page by page: space for the next page, "q" to quit. In Colab, download the two files from the work folder under "/tmp" instead. The work folder stays in your temp folder; the system clears it in time.

You Can Now

Compute Macaulay duration by hand, as a value-weighted average time, and in a sheet with "SUMPRODUCT".

Write a test first and watch it fail before the function exists.

Compute Macaulay and modified duration with "bondmath", tied to "DURATION" and "MDURATION".

Turn modified duration into DV01 per 100, per position, and for the book, in dollars.

Check DV01 by bumping the yield 1 bp either way.

Read the sign of DV01 for a long and a short position.

Ask Git which commits changed one file, and what the last one added.

Predict first: the average time, and a higher yield

Predict 1

The last payment is 4.833333 years away and carries 79 percent of the value; nine coupons come earlier. What is the value-weighted average time of the ten payments, in years? Write down a number.

```
macaulay_by_hand = (flows['time_years'] * flows['weight']).sum()
print(f'Weighted average time: {macaulay_by_hand:.6f} years')
print(f"Last payment:          {flows['time_years'].iloc[-1]:.6f} years")
```

Predict 2

Give Quorvane's dates a coupon of 0: a zero, one payment at maturity. Move Quorvane and the zero from 5.40 to 10 percent. Their Macaulay durations:

A) both fall

B) Quorvane's falls, and the zero's does not move

C) neither moves

```
for yield_pct in (5.40, 10.0):
    coupon_bond = bondmath.macaulay_duration(settlement, quorvane_maturity, 5.25, yield_pct)
    zero = bondmath.macaulay_duration(settlement, quorvane_maturity, 0.0, yield_pct)
    print(f'At {yield_pct:>5.2f} percent: Quorvane {coupon_bond:.6f}, the zero {zero:.6f} years')
```

Predict first: positions, history, the column

Predict 3

Which position carries the most dollars per basis point?

A) Halvering, the largest DV01 per 100

B) Quorvane, the largest position

C) Dunmarrow, second on both

```
bonds['position_dv01'] = bonds['dv01'] * bonds['par_held'] / 100
print(f"Book DV01: {bonds['position_dv01'].sum():,.2f} dollars per basis point")
bonds[['issuer', 'position_dv01']].round(2)
```

Predict 4

Nine commits: one per notebook, and today's. "git log --oneline -- bondmath.py" lists only those that changed "bondmath.py". How many? Write down a number.

```
!git -C "{work}" log --oneline -- bondmath.py
```

Predict 5

If the column is modified duration, Macaulay sits above it by modified duration x y / 2; the file's longest is 16.0 years at 5.2 percent. At worst, Macaulay misses the file by:

A) about 0.5 years

B) about 0.05 years

C) about 0.005 years

```
for column in ['macaulay_python', 'modified_python']:
    largest = (holdings[column] - holdings['duration']).abs().max()
    print(f'Largest gap from the file, {column}: {largest:.6f} years')
```

The note, answered

The note asked	Answer
The book	6,154.97 dollars per bp
Duration and DV01 tied to Excel	macaulay_duration, modified_duration, dv01, all 306 rows
The file's column	Modified duration, all 200 bonds within rounding

Be proud: you can say where a risk number comes from, down to each cash flow, and which way its sign points.

Next: Case Study: Convexity and the September Book (notebook 10) measures what the straight line misses.